Software Architecture and Design Specification

Project: Task Manager Application
Version: 1.0
Authors: <Task Manager Project Team>
Date: 30-09-2026
Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	30-09-2026	Project Team	Initial architecture and design

Approvals

Role	Name	Signature / Date
Architect / Tech Lead		
Project Guide		
QA Lead		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Task Manager Application, a tool to organise and track work progress.

1.2 Scope

Covers user services: account management, task and project management, assignment, search/filter, Kanban board and dashboard, reminders, comments, activity history, CSV export and user administration.

1.3 Audience

Developers, QA engineers, security reviewers, instructors and maintenance teams.

1.4 Definitions

SPA (Single-Page Application), REST, JWT, RBAC, ORM, ADR (Architecture Decision Record), STRIDE, TLS, API, CRUD.

2. Document Overview

2.1 How to use this document

Provides the architectural deliverables: component (UML) diagram, architecture decision records, technology stack, security threat model, UML sequence diagrams and API design. It should be read together with the SRS and the Test Plan.

2.2 Related Documents

Software Requirements Specification (SRS v1.0), Software Test Plan (STP v1.0), Requirements Traceability Matrix (RTM, SRS Section 8).

3. Architecture

3.1 Goals & Constraints

- **Goals:** simple and fast task handling, secure user data, 99.5% availability, easy maintenance, and support for 1,000 concurrent users.
- **Constraints:** open-source stack, browser-only client, OWASP-aligned security practices, delivery within the academic timeline.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
End users (Members)	Usability, speed, reliable reminders, data privacy
Project Owners / Team Leads	Clear progress tracking, access control
Administrators / Ops	Maintainability, monitoring, backups
Developers	Modularity, testability, clear interfaces
QA / Evaluators	Traceability to requirements, testability

3.3 Component (UML) Diagram

The system is organised into four layers. Each layer only calls the layer directly beneath it.

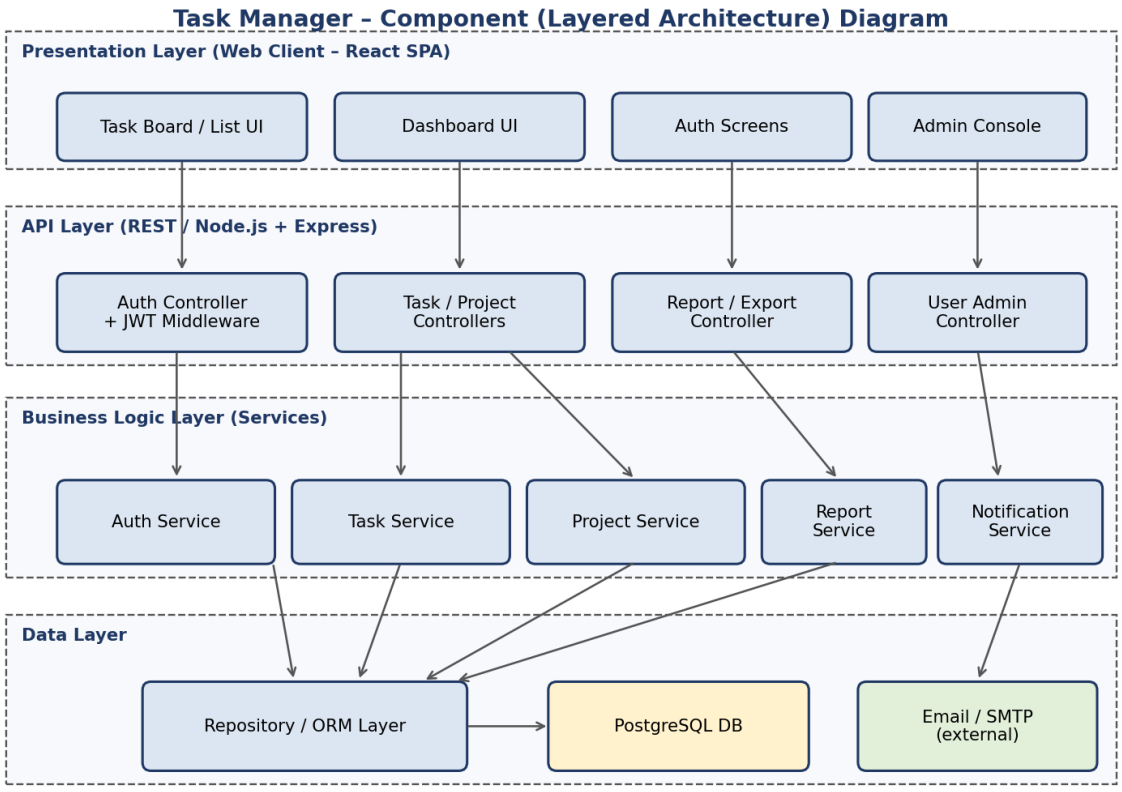


Figure 1 - Component Diagram (Layered Architecture)

3.4 Component Descriptions

Component	Responsibility
Web Client (React SPA)	Renders task list, Kanban board, dashboard, auth screens and admin console; calls the REST API.
Auth Controller + JWT Middleware	Handles register/login/reset endpoints; verifies tokens and roles on every protected request.
Task / Project Controllers	Expose CRUD, search, assignment, comment and project endpoints; validate input.
Report / Export Controller	Provides dashboard statistics, activity history and CSV export.
Auth Service	Password hashing (bcrypt), token issue/refresh, lock-out policy, account management.
Task Service	Business rules for tasks, status transitions, assignment, comments, activity logging.
Project Service	Project membership, categories/tags and project-level access checks.
Report Service	Aggregations for dashboard, task history and export.
Notification Service	Schedules and sends in-app and email reminders and password-reset mails.
Repository / ORM Layer	Database access through parameterised queries; hides SQL from services.
PostgreSQL Database	Persistent storage of users, projects, tasks, comments, activity and audit logs.

3.5 Chosen Architecture Pattern and Rationale

A **layered (n-tier) client-server architecture** with a stateless REST API was chosen because it separates concerns (UI, API, business logic, data), is easy for a small team to develop and test, and scales horizontally by running several identical API instances behind a load balancer. Microservices were rejected as overly complex for the expected scale and team size.

ADR	Decision	Alternatives considered	Reason
ADR-01	Layered monolithic backend	Microservices	Lower operational complexity; sufficient for scale
ADR-02	React single-page application	Server-rendered pages	Rich interactions (drag-and-drop Kanban), responsive UI
ADR-03	PostgreSQL relational DB	MongoDB	Strong relations between users, projects, tasks; ACID transactions
ADR-04	Stateless JWT authentication	Server-side sessions	Simple horizontal scaling; short expiry plus refresh token

3.6 Technology Stack & Data Stores

Area	Technology
Frontend	React, TypeScript, CSS modules; drag-and-drop library for Kanban
Backend	Node.js with Express, REST/JSON, JWT (jsonwebtoken), bcrypt
Database	PostgreSQL 15 with an ORM (Prisma / Sequelize)
Scheduler / Email	node-cron job runner, Nodemailer over SMTP
Security	HTTPS/TLS 1.2+, Helmet, express-rate-limit, input validation library
DevOps	Git, GitHub Actions CI, Docker; Jest, Cypress, JMeter for testing

Main data entities:

Table	Key fields
users	id, name, email (unique), password_hash, role, failed_attempts, locked_until, active
projects	id, name, owner_id, archived, created_at
project_members	project_id, user_id, member_role

Table	Key fields
tasks	id, project_id, title, description, due_date, priority, status, category, assignee_id, created_by, completed_at, deleted_at
tags / task_tags	id, name / task_id, tag_id
comments	id, task_id, author_id, body, created_at
activity_log	id, task_id, user_id, action, old_value, new_value, created_at
audit_log	id, user_id, event_type, ip, created_at
reminders	id, task_id, user_id, send_at, channel, sent

3.7 Risks & Mitigations

Risk	Mitigation
Database outage or data loss	Daily backups, restore drills, connection retry with pooling
Slow queries as task volume grows	Indexes on project_id, assignee_id, status, due_date; pagination
Reminder emails not delivered	Retry queue with back-off; in-app reminder as fallback
Security vulnerabilities (injection, XSS, broken access control)	Validation, parameterised queries, server-side RBAC, ZAP scans in CI
Scope creep in a short project	Prioritised requirements (High first); Low items deferred

3.8 Traceability to Requirements

Requirement	Component(s)
TM-F-001 to 004 (Authentication)	Auth Controller, Auth Service, Notification Service
TM-F-010 to 014 (Task management)	Task Controller, Task Service, Notification Service
TM-F-020 to 023 (Projects, search, reminders)	Project Service, Task Service, Notification Service
TM-F-030 to 032 (Kanban, dashboard, history)	Task Board / Dashboard UI, Report Service
TM-F-040 to 043 (Comments, admin, export, access)	Task Service, Auth Service, Report Service, Project Service
TM-NF-001, 005 (Performance, scalability)	Indexed DB, stateless API instances, pagination
TM-NF-002, 007 (Availability, recovery)	Deployment monitoring, backup jobs
TM-SR-001 to 007 (Security)	Gateway/TLS, Auth Service, JWT middleware, validation layer, audit log

3.9 Security Architecture

Threat modelling with STRIDE:

Threat	Example	Mitigation
Spoofing	Stolen or guessed credentials	bcrypt hashes, lock-out after 5 failures, rate limiting, short-lived JWT
Tampering	Modifying another user's task via API	Server-side RBAC and project-membership checks, input validation
Repudiation	User denies deleting a task	Activity log and audit log with user, time, IP
Information disclosure	Sniffing traffic, leaking passwords in logs	TLS 1.2+, no secrets in logs, generic error messages
Denial of service	Flooding login or API	Rate limiting, request size limits, pagination
Elevation of privilege	Member calls admin endpoint	Role claims verified on each request; admin routes separated

4. Design

4.1 Design Overview

The backend is designed as layered modules: controllers (HTTP concerns), services (business rules), and repositories (data access). Dependencies point downward only, so each service can be unit-tested with mocked repositories. The frontend is organised by feature (auth, tasks, board, dashboard, admin) with a shared API client and state store.

4.2 UML Sequence Diagrams

Two key flows are shown: user login (authentication) and creation of a task with reminder scheduling.

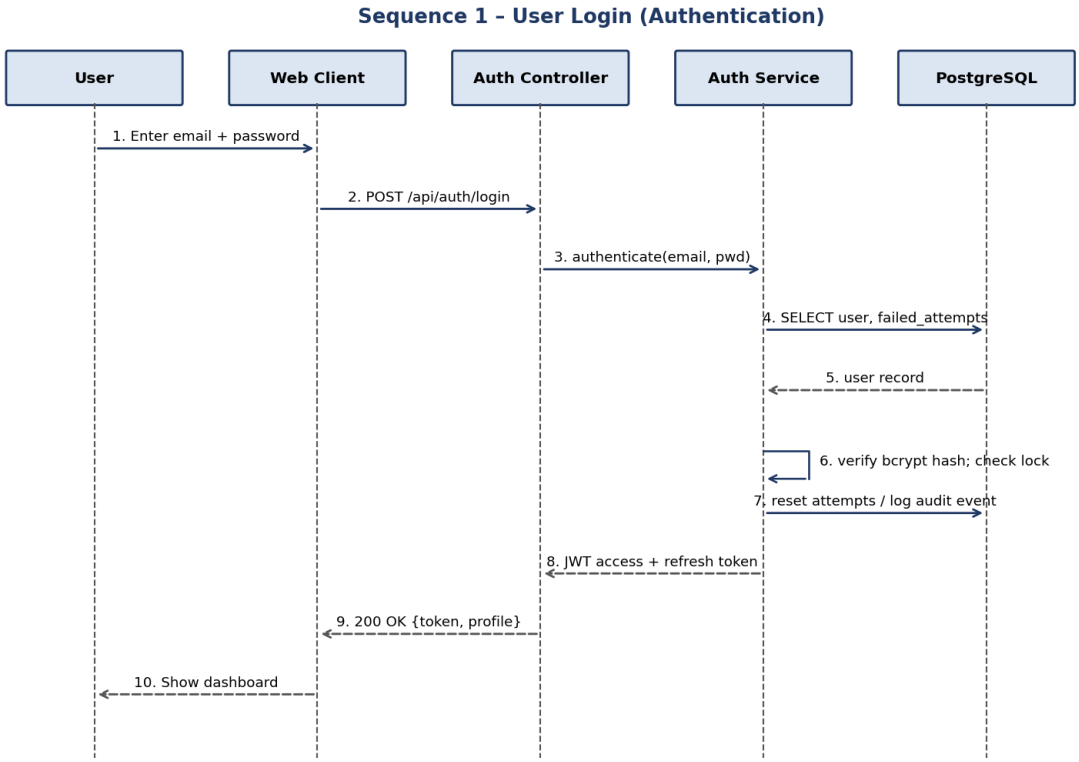


Figure 2 - Sequence Diagram: User Login

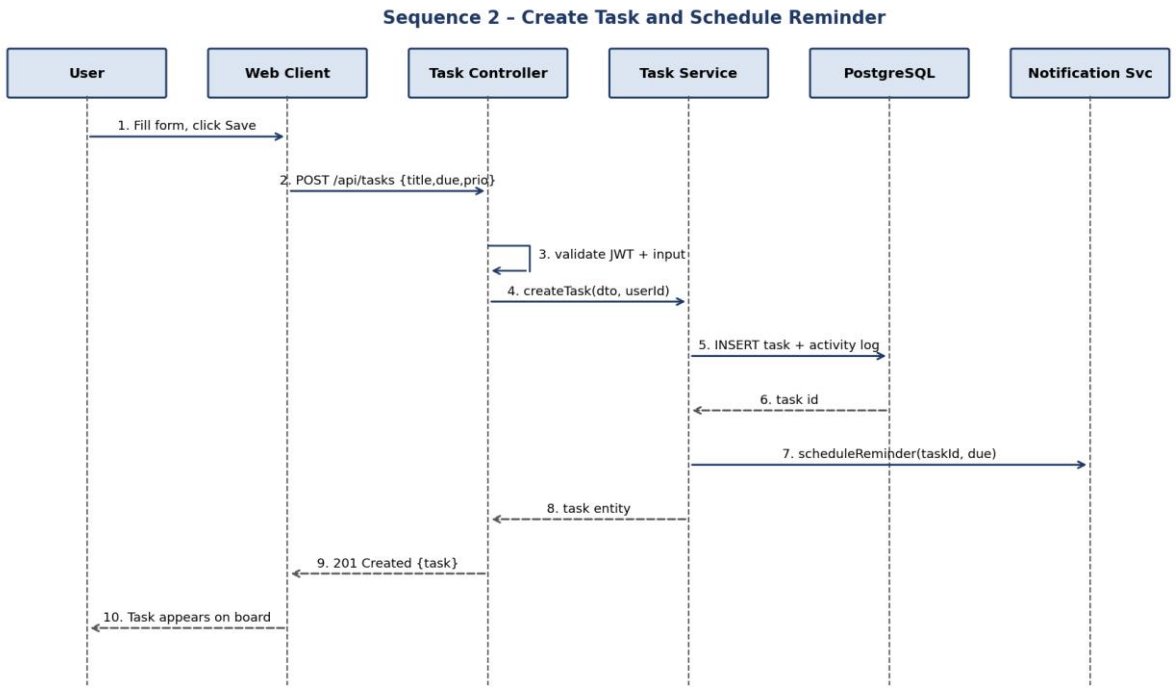


Figure 3 - Sequence Diagram: Create Task and Schedule Reminder

4.3 API Design

All endpoints use JSON over HTTPS. Protected endpoints require the header Authorization: Bearer <token>.

Component 1 - Auth API

Endpoint	Method	Request	Response	Errors
/api/auth/register	POST	{name, email, password}	201 {userId}	400 Invalid data, 409 Email exists
/api/auth/login	POST	{email, password}	200 {accessToken, refreshToken, profile}	401 Invalid credentials, 423 Account locked, 429 Too many requests
/api/auth/logout	POST	{refreshToken}	204	401 Unauthorised
/api/auth/reset	POST	{email} or {token, newPassword}	202 / 200	400 Invalid or expired token

Component 2 - Task API

Endpoint	Method	Request	Response	Errors
/api/tasks	POST	{projectId, title, description, dueDate, priority, assigneeId?}	201 {task}	400 Validation error, 401, 403 Not a member
/api/tasks?status=&priority=&q=&sort=	GET	Query parameters	200 {items[], page, total}	401, 403
/api/tasks/{id}	PATCH	Changed fields, e.g. {status:"DONE"}	200 {task}	400, 403, 404 Not found
/api/tasks/{id}	DELETE	-	204 (soft delete)	403, 404
/api/tasks/{id}/comments	POST	{body}	201 {comment}	400, 403, 404
/api/reports/dashboard	GET	projectId (optional)	200 {countsByStatus, percentComplete, overdue}	401, 403
/api/tasks/export.csv	GET	Same filters as list	200 text/csv	401, 403

4.4 Error Handling, Logging & Monitoring

- Standard error body: {code, message, details?}; generic messages for authentication failures; HTTP status codes used consistently.
- Structured logs with request id; passwords, tokens and personal data are never logged.
- Monitoring: /health endpoint, uptime checks, error rate, response-time percentiles, failed-login rate and reminder-delivery failures.

4.5 UX Design

Clean layout with a project sidebar, quick-add task field, colour-coded priority labels and an overdue highlight. Kanban cards support drag-and-drop with keyboard alternatives. Responsive breakpoints for tablet and mobile, accessible font sizes and contrast (WCAG 2.1 AA), and an optional dark theme.

4.6 Open Issues & Next Steps

- Future enhancements: file attachments, recurring tasks, calendar integration, mobile app, real-time updates through WebSockets.
- Decide the final ORM (Prisma vs Sequelize) in the first sprint.
- Finalise email provider and reminder schedule defaults.

5. Appendices

5.1 Glossary: SPA, REST, JWT, RBAC, ORM, ADR, STRIDE, TLS, CRUD.

5.2 References: IEEE 42010, OWASP Top 10 / ASVS, NIST SP 800-160, Task Manager SRS v1.0.

5.3 Tools: PlantUML, draw.io, Swagger / OpenAPI, Postman, GitHub.